LAB7 – Cloud

LAB — MICROSERVICOS NA AWS

Voce foi contratado para criar a mesma API de saudação do exercício anterior, mas desta vez **como um Microservico com CONTAINER**. Como o tráfego e previsível e a equipe precisa de controle total sobre o ambiente de execução, a arquitetura deve ser baseada em **containers Docker** implantados no **Amazon ECS com Fargate**, eliminando a gestão de servidores sem abrir mão da flexibilidade.

Diferença-chave em relação ao Serverless:

No Lambda, você só escreve a função. Aqui, você empacota a aplicação inteira num container Docker e a AWS gerencia onde e como ele executa — isso e a essência dos Microservicos gerenciados.

Vamos as dicas...

Acessar o Console AWS e abrir o CloudShell

- Faça login no console da AWS (**console.aws.amazon.com**).
- Clique no ícone do **CloudShell** (> _) no canto superior direito da barra de navegacao.
- Aguarde o terminal carregar — ele já tem o **AWS CLI** e o **Docker** pré-instalados.

O que e o CloudShell?

E um terminal Linux diretamente no navegador, sem nenhuma instalação local.

Você o usara para criar o código, o Dockerfile e enviar a imagem para a AWS.

Criar o Repositório no Amazon ECR

- Na barra de busca do console, digite **ECR** e selecione **Elastic Container Registry**.
- **Clique em Create repository**.
- Em **Repository name**, digite:

```
microservico-SEUNOME
```

- Deixe as demais opções no padrão e clique em **Create repository** (botão laranja).
- Anote o **URI do repositório** que aparece na lista. Ele tem o formato:

```
123456789012.dkr.ecr.us-east-1.amazonaws.com/microservico-SEUNOME
```

Criar o Código da Aplicação no CloudShell

- Volte ao terminal **CloudShell** e crie a pasta do projeto:

```
mkdir projeto && cd projeto
```

- Crie o arquivo da aplicacao Python (Flask) com o comando abaixo — ele abrira o editor nano:

```
nano app.py
```

- Cole o seguinte código no editor (não se preocupe — e simples!):

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/', methods=['GET'])
def msg():
    # Extrai o nome do corpo da requisicao
    dados = request.get_json() or {}
    nome = dados.get('nome', 'Visitante')

    # Cria a mensagem de saudacao
    mensagem = f'Ola, {nome}! Bem-vindo ao mundo dos Microservicos.'

    # Retorna a resposta em JSON
    return jsonify({'statusCode': 200, 'body': mensagem})

@app.route('/health')
def health():
    return jsonify({'status': 'ok'})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080)
```

- Pressione **Ctrl+X**, depois **Y** e **Enter** para salvar.

Criar o Dockerfile

- Ainda no CloudShell, crie o Dockerfile:

```
nano Dockerfile
```

- Cole o seguinte conteudo:

```
FROM python:3.9-slim

WORKDIR /app

# Instala a dependencia Flask
RUN pip install flask

# Copia o codigo da aplicacao
COPY app.py .

# Expoe a porta 8080
EXPOSE 8080

# Comando para iniciar a aplicacao
CMD ["python", "app.py"]
```

- Salve com **Ctrl+X**, **Y**, **Enter**.

Construir e Publicar a Imagem Docker no ECR

- No CloudShell, execute os comandos abaixo, **um por vez**. Substitua SEU-ACCOUNT-ID e SEUNOME pelos valores corretos:

```
# 1. Autenticar o Docker no ECR
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS \
  --password-stdin SEU-ACCOUNT-ID.dkr.ecr.us-east-1.amazonaws.com

# 2. Construir a imagem Docker já com o endereço do ECR
docker build -t SEU-ACCOUNT-ID.dkr.ecr.us-east-
1.amazonaws.com/microservico-SEUNOME:latest .
```

```
# 3. Enviar a imagem para o ECR
docker push \
  SEU-ACCOUNT-ID.dkr.ecr.us-east-1.amazonaws.com/saudacao-microservico-
SEUNOME:latest
```

- Aguarde a conclusao. Voce vera a mensagem **latest: digest: sha256:...** ao final, confirmando o envio.

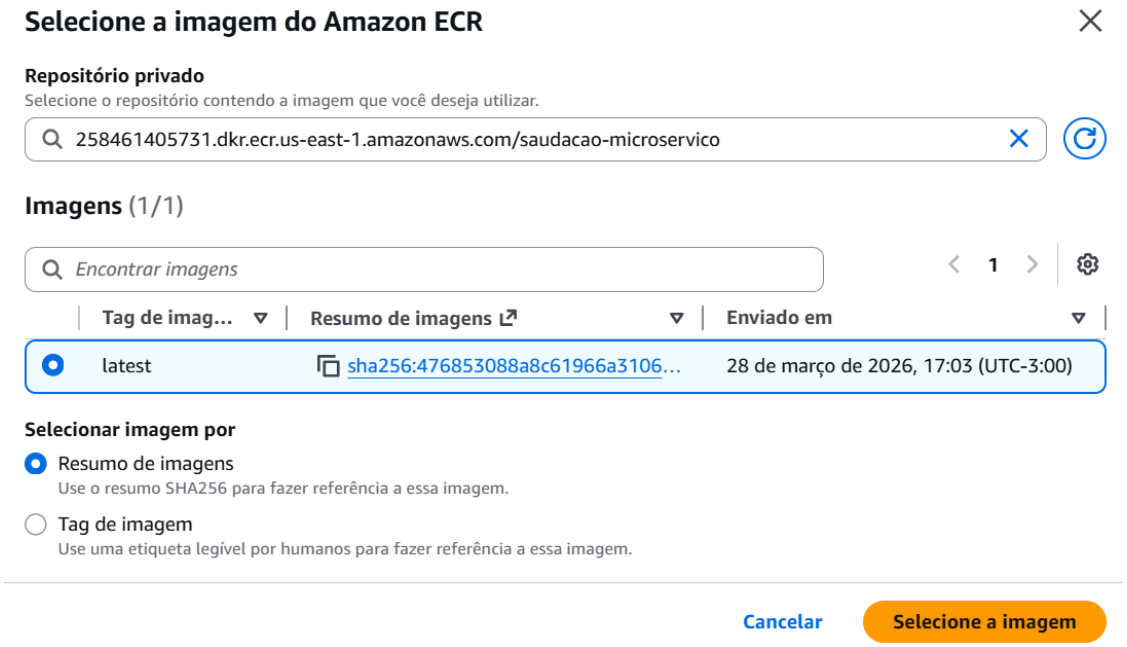
ATENÇÃO

Até o momento criamos uma IMAGEM para instalação de um container (servidor virtual) para utilização da nossa aplicação. Podemos fazer essa imagem com qualquer aplicação, desde java, node, python, spring, etc...

IMPLANTNDO O MICROSERVIÇO NA AWS

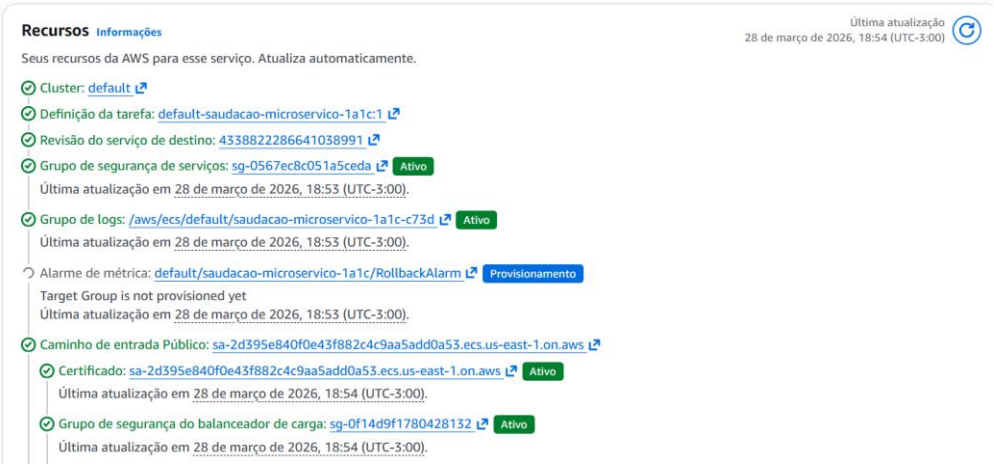
Criar o Cluster no Amazon ECS

- Na barra de busca do console, digite **ECS** e selecione **Elastic Container Service**.
- **Clique em Express Mode**.
- Localize a imagem que você carregou para **Repository**: `microservico-SEUNOME`.
- Selecione a versão



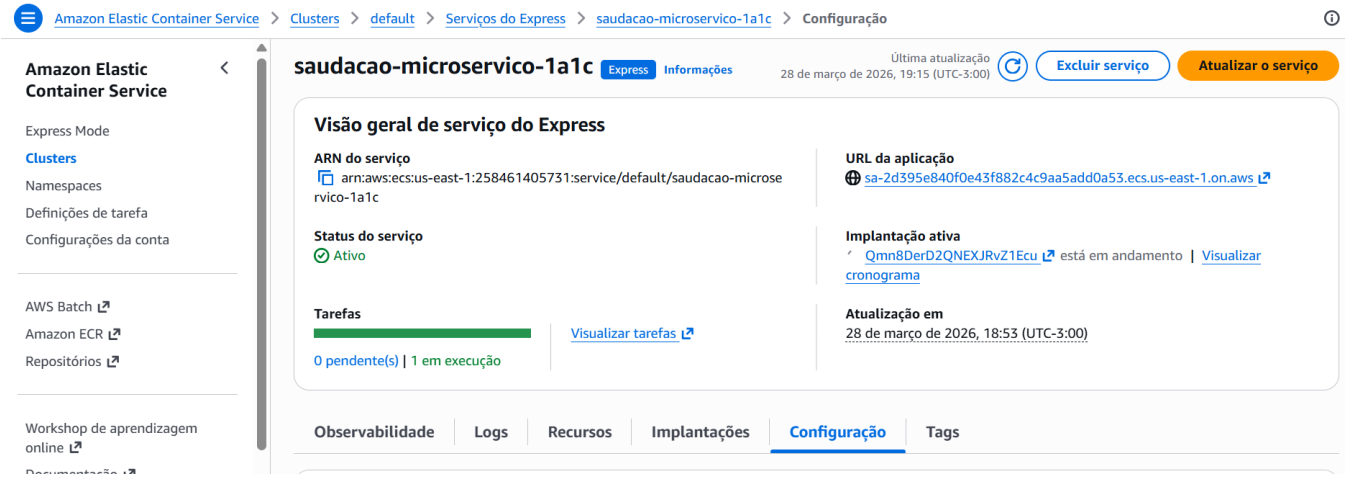
- Função de Execução e Perfil da Infraestrutura: escolha o **LabROLE**
- **Clique em CRIAR**

AGUARDE FINALIZAR

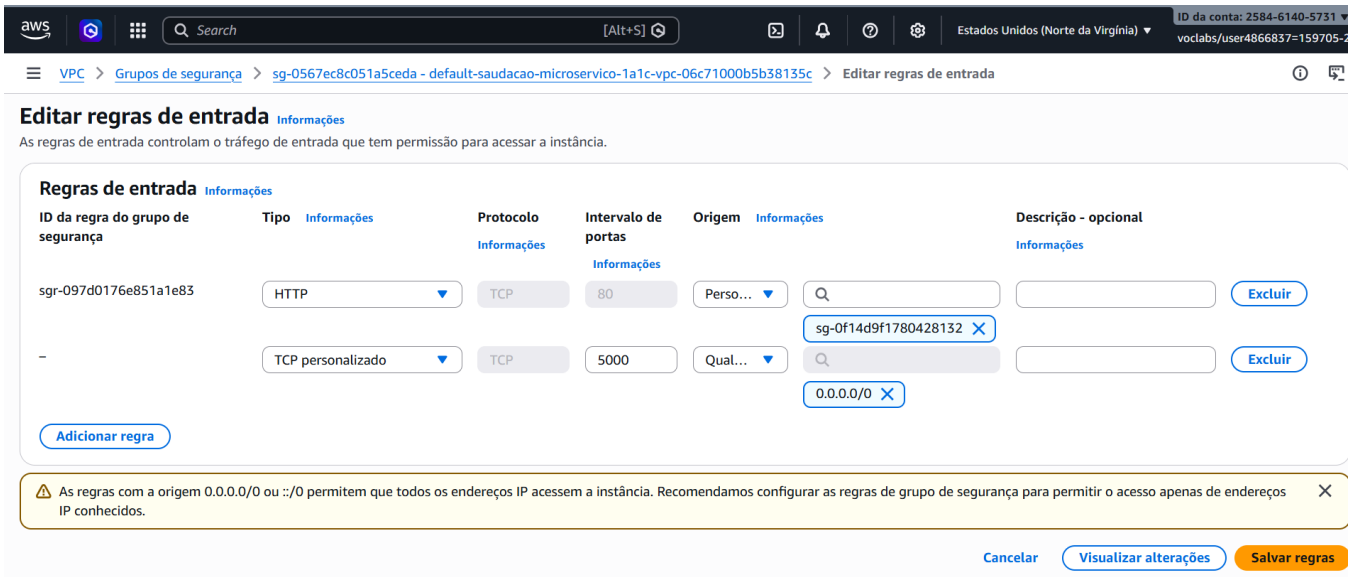


Selecione o CLUSTER

- No menu lateral do ECS, clique em **Cluster**.
- **Selecione o cluster "default"**.
- Agora precisamos liberar a porta 8080 utilizada pelo nosso sistema
- Selecione o microserviço ativo e depois Configuração



- Localizar o Grupo de Segurança
- E adicione a porta 8080



Testar o Microservico

- No cluster, clique na aba **Tasks** e selecione a task em execução.
- Na seção **Configuration**, copie o valor de **Public IP**.
- Volte ao **CloudShell** e teste a API com o comando abaixo (substitua IP-PUBLICO pelo IP copiado):

```
curl -X POST http://IP-PUBLICO:8080/ \
  -H 'Content-Type: application/json' \
  -d '{"nome": "Coloque aqui seu nome"}'
```

Ou apenas abrir o browser com o endereço `http://IP-PUBLICO:8080/`

- Voce devera ver a resposta:

```
{
  "statusCode": 200,
  "body": "Ola, XXXXX! Bem-vindo ao mundo dos Microservicos."
}
```

Analisar o Resultado

- Observe os detalhes da Task no console ECS:
  - **Status**: RUNNING — o container esta em execução ativa.
  - **CPU / Memory usage**: na aba **Metrics** do serviço.
  - **Logs**: clique em **Logs** dentro da task para ver as requisições recebidas.
- Compare com o Lambda do exercício anterior:
  - **Lambda**: acordou, executou e dormiu — você pagou apenas pelo tempo de execução.
  - **ECS Fargate**: o container fica rodando continuamente, pronto para responder a qualquer momento.

Entregável

- Tire um **print screen** do terminal CloudShell mostrando a resposta da API com a mensagem de sucesso e seu nome.

Não esqueça de parar o serviço ao terminar!

No cluster > Services > selecione o serviço > Update > Desired tasks: 0.

Isso evita custos desnecessários enquanto o exercício não está em uso.

Desafio Extra (EAD)

Modifique o código para receber também a cidade no JSON de entrada e retornar uma mensagem personalizada incluindo a cidade de origem do usuário.

- Exemplo de entrada: **{"nome": "Ana", "cidade": "São Paulo"}**
- Exemplo de saída esperada: **"Ola, Ana, de São Paulo! Bem-vindo ao mundo dos Microservicos."**

- Apos modificar o **app.py**, refaça os passos (build + push) e atualize o serviço ECS **(Update Service > Force new deployment)** para que a nova versão seja implantada.
- Tire um **print screen** da resposta da API mostrando a mensagem com nome e cidade.